# Lab IV
# (Diseño Circuito Combinacionales)

Alex Rafael Soriano Severino
Mecatronica
ITLA
La Caleta, Santo Domingo
20240684@itla.edu.do

**En resumen— En esta práctica se diseñó, analizó e implementó un sistema combinacional capaz de realizar operaciones aritméticas básicas de suma, resta y multiplicación entre dos operandos de 4 bits, y además se integró un comparador de 5 bits que permite determinar si el operando A es mayor, menor o igual que B. El objetivo fue aplicar los conceptos de análisis y síntesis lógica, diseño modular y estructural en Verilog, e implementación en FPGA utilizando un multiplexor (MUX) como elemento de control para la selección de la operación o la comparación.**

**Se elaboraron las tablas de verdad correspondientes a cada bloque funcional, se obtuvieron las funciones booleanas mínimas mediante mapas de Karnaugh, y se construyeron los diagramas lógicos teóricos en Multisim. Posteriormente, se desarrolló el diseño estructural en Verilog, instanciando los submódulos (sumador, restador, multiplicador, comparador y MUX) dentro de un módulo principal. Finalmente, se verificó el funcionamiento correcto del sistema sobre la FPGA, confirmando la coherencia entre el diseño teórico, la simulación digital y los resultados prácticos.**

**Abstract— In this practice, a 4-bit arithmetic system capable of performing addition, subtraction, and multiplication operations, as well as a 5-bit comparator, was designed and implemented. The goal was to apply combinational logic design principles, hierarchical structural modeling in Verilog, and FPGA synthesis using a multiplexor for operation selection. Truth tables and Karnaugh maps were used to obtain minimized Boolean functions for each block. Logical diagrams were built in Multisim, and the structural Verilog design was implemented using modular instantiation. The circuit was synthesized and tested on an FPGA board, demonstrating functional equivalence between theoretical and experimental results.**

***Palabras claves*— lógica combinacional, sumador, restador, multiplicador, comparador, multiplexor, Verilog estructural, FPGA, simulación digital.**

## I. INTRODUCCION

En este laboratorio se desarrolló un circuito combinacional aritmético de 4 bits que permite realizar tres operaciones fundamentales: suma, resta y multiplicación. En esta segunda parte, se añadió un comparador de 5 bits para extender la funcionalidad del sistema, permitiendo determinar la relación entre dos operandos (A > B, A = B o A < B).

Cada una de estas funciones fue implementada mediante el diseño de bloques lógicos básicos y posteriormente integradas bajo una arquitectura modular controlada por un multiplexor. El propósito principal fue reforzar los conocimientos de álgebra booleana, mapas de Karnaugh, y diseño jerárquico en Verilog, comprendiendo el proceso completo desde la teoría hasta la implementación física en FPGA.

El proyecto incluyó las etapas de:

1. Análisis lógico y simplificación de funciones.
2. Modelado teórico mediante diagramas lógicos en Multisim.
3. Codificación estructural en Verilog.
4. Síntesis y verificación del diseño sobre la FPGA. Este laboratorio permitió comprender cómo un sistema digital puede ejecutar distintas operaciones aritméticas y de comparación dentro de un mismo circuito, utilizando estrategias de reutilización de hardware y control lógico.

## II. MARCO TEÓRICO

### *A.* Circuitos Aritméticos Combinacionales

Un circuito aritmético combinacional es aquel que realiza operaciones matemáticas utilizando únicamente compuertas lógicas. A diferencia de los circuitos secuenciales, su salida depende exclusivamente de las entradas actuales. Entre los más comunes se encuentran el sumador, el restador y el multiplicador binario.

### *B.* Sumador de 4 bits

El sumador binario combina pares de bits (A y B) generando una salida de suma y un acarreo (carry). Un sumador completo (Full Adder) puede implementarse mediante compuertas XOR, AND y OR, y su concatenación permite obtener un sumador de 4 bits.

### *C.* Restador de 4 bits

La resta binaria puede realizarse mediante la suma del complemento a dos del sustraendo. Así, un restador de 4 bits puede implementarse reutilizando el mismo bloque de sumadores, invirtiendo las entradas y controlando el bit de acarreo inicial.

### *D.* Multiplicador de 4 bits

El multiplicador combina operaciones AND y sumadores parciales para generar un producto de hasta 8 bits. Este bloque suele construirse a partir de sumadores de medio y completo, organizados en forma matricial.

### *E.* *Comparador de 5 bits*

El comparador binario es un circuito combinacional que recibe dos palabras binarias (A y B) y determina su relación

lógica. Su salida indica si A es mayor que B, menor o igual. En este caso, se implementó un comparador de 5 bits, lo cual permite ampliar el rango de comparación de los operandos.

### *F.* Multiplexor (MUX)

El multiplexor es un dispositivo de selección que permite dirigir una de varias señales de entrada hacia una única salida. En este proyecto, el MUX se empleó para elegir entre la suma, resta o multiplicación, controlado mediante dos bits de selección (S1, S0).

### *G.* Diseño estructural en Verilog

La descripción estructural consiste en instanciar módulos más pequeños dentro de un módulo principal, de forma análoga a conectar compuertas lógicas. Este enfoque refuerza la noción de diseño jerárquico y modular, esencial para implementaciones en FPGA.

## III. Montaje Experimental

El desarrollo del laboratorio se realizó en tres fases principales:

1. **Diseño lógico y síntesis:**
   - Se elaboraron las tablas de verdad para las operaciones de suma, resta, multiplicación de 4 bits y comparación.
   - Se simplificaron las funciones utilizando mapas de Karnaugh, obteniendo las expresiones mínimas de cada bit de salida.
   - Se diseñó el multiplexor de selección de operación, con dos líneas de control que determinan la salida activa (suma, resta o multiplicación).
2. **Simulación teórica en Multisim:**
   - Cada bloque funcional fue implementado a nivel de compuertas (AND, OR, XOR, NOT).
   - Se verificó el comportamiento lógico de cada operación y la correcta selección de salida mediante el MUX.
   - Los resultados se capturaron en diagramas lógicos para incluirlos en el informe.
3. **Implementación estructural en Verilog:**
   - Se codificaron los módulos: full_adder, sumadora, restadora, multiplicadora, y mux 3 a 1.
   - Todos los módulos fueron conectados dentro de un módulo principal (MAIN) que dirige las señales de entrada y salida.
   - Se evitó el uso de declaraciones *assign*, priorizando la instanciación de compuertas y módulos.
   - El código fue sintetizado en GoWin IDE y cargado en la FPGA, verificando su funcionalidad con operandos binarios de 4 bits.

## IV. Funcionamiento del Circuito:

El sistema recibe dos operandos de 4 bits (A[3:0], B[3:0]) para las operaciones aritméticas, y de 5 bits para el módulo comparador. Dos bits de control (S1, S0) determinan la operación o comparación a ejecutar:

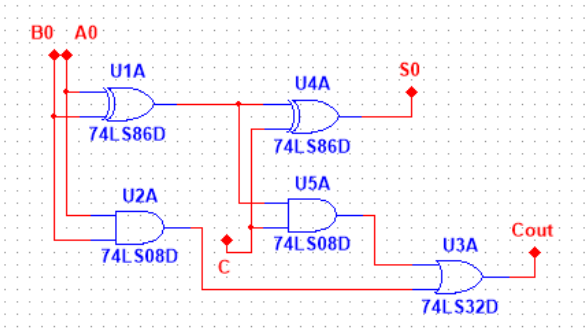
| S1 | S2 | Operación |
|---|---|---|
| 0 | 0 | Suma (A + B) |
| 0 | 1 | Resta (A - B) |
| 1 | 0 | Multiplicación (A × B) |
| 1 | 1 | Comparación (A ? B) |

La salida del circuito es un bus binario de 8 bits (Resultado[7:0]) que se muestra en los LEDs de la FPGA. Durante las pruebas, se verificó que:

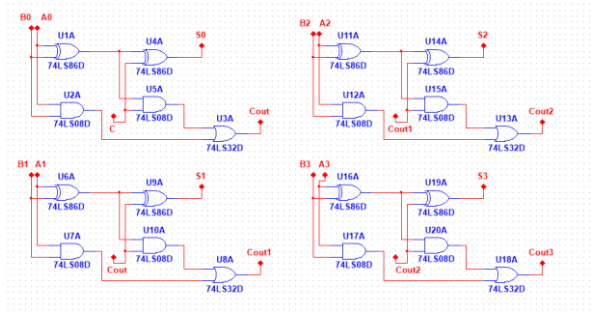
- El comparador mostró correctamente las tres condiciones (mayor, igual, menor).
- El MUX seleccionó adecuadamente entre las salidas de las operaciones y la del comparador.
- Todas las operaciones respondieron coherentemente a las entradas y señales de control.

**Diseño lógico (MULTISIM):**

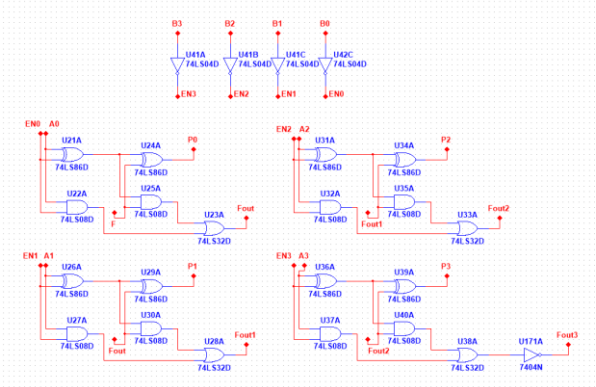
**Fulladder:**



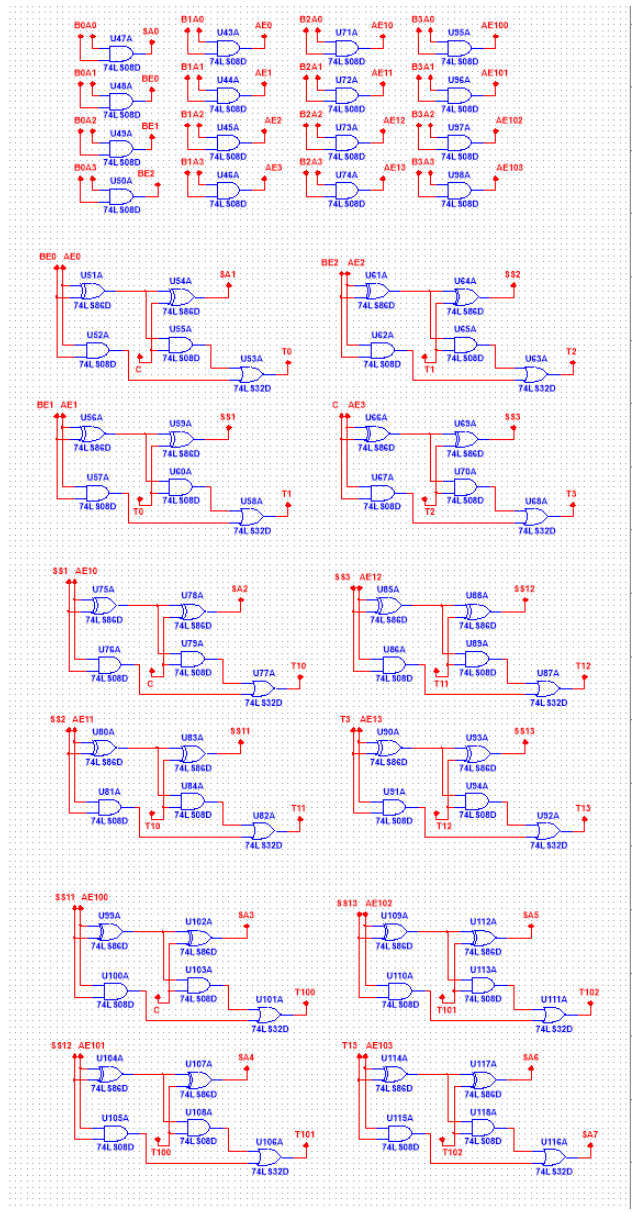

**Adder(Sumadora):**



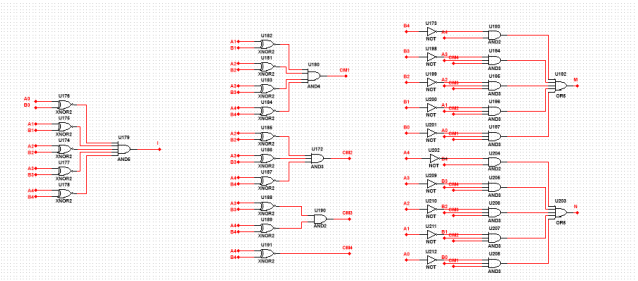
**Subtractor(Restadora):**



**Multiplier(Multiplicadora):**



**Comparator (Comparador):**



**MUX4TO1(Multiplexor 4:1):**

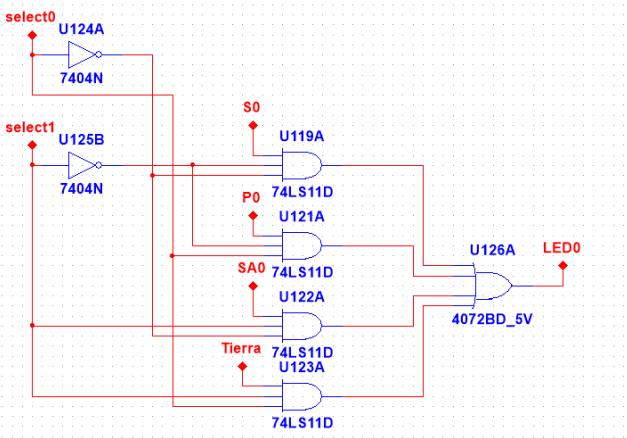



**Diagrama de MUX para las salidas:**

Aquí se utilizaron varios MUX de este modelo en concreto se utilizaron 8, 1 para cada bit de salida, así fue como se implemento un solo bus de salida de leds.

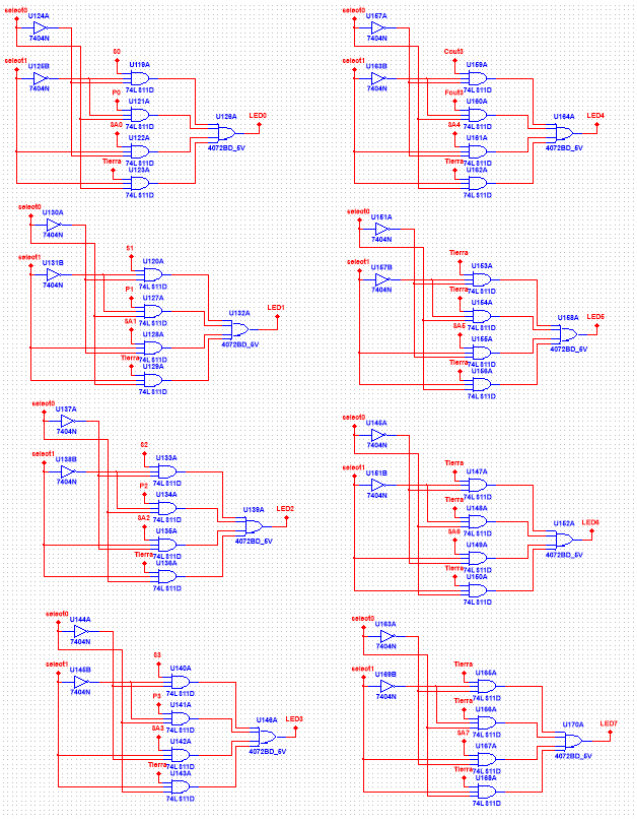
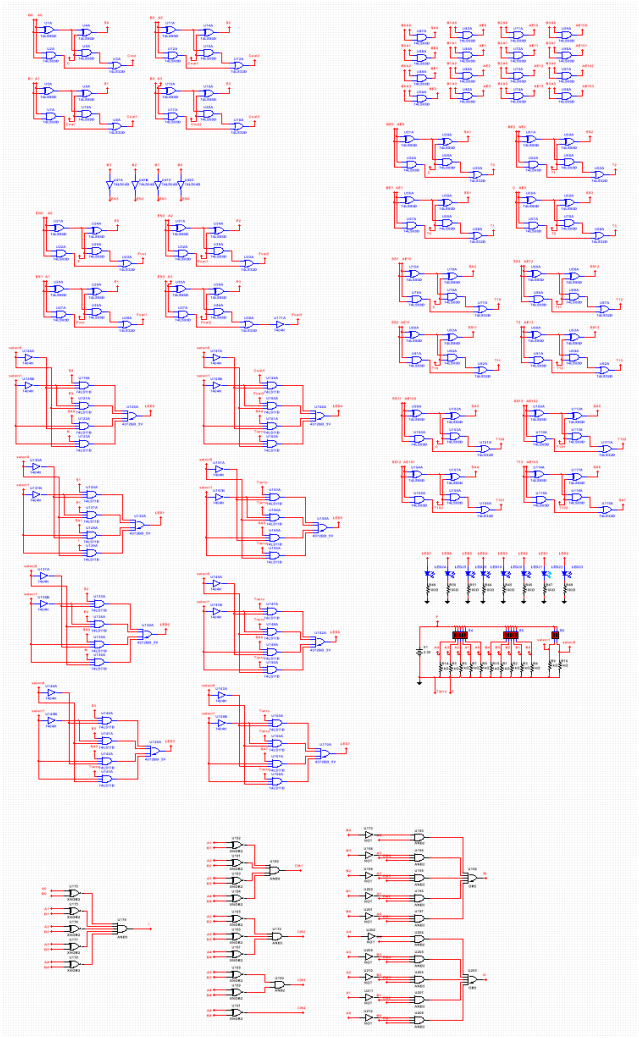
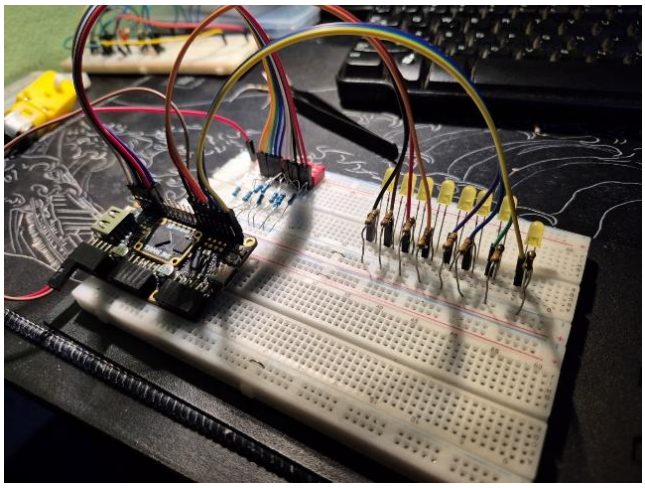


**Diagrama del circuito completo:**



**Físico:**



## V. ANÁLISIS Y/O CÁLCULOS

Aquí calcularemos todos los datos solicitados en la práctica:

### A. Tablas de Verdad Teórica

Esta es la tabla de una fulladder, para la sumadora lo que haremos es concatenar 4 fulladder para crear una sumadora de 4 bits, así no tenemos que crear una taba tan grande para la sumadora.
En cuanto a la restadora usaremos una sumadora modificada para obtener la resta, lo haremos con el método de complementos a 2 en el segundo numero (B).
Para la multiplicadora hacemos las multiplicaciones parciales con compuertas ands y luego haremos las sumas corriendo el bit de acarreo un espacio hacia la izquierda para obtener la suma de cada producto, esto es básicamente el mismo modelo que hacemos cuando calculamos el producto de dos números binarios en papel.

| | Tabla de Verdad | | | | |
|---|---|---|---|---|---|
| | A | B | C | Cout | S |
| 1 | 0 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 1 | 0 | 1 |
| 3 | 0 | 1 | 0 | 0 | 1 |
| 4 | 0 | 1 | 1 | 1 | 0 |
| 5 | 1 | 0 | 0 | 0 | 1 |
| 6 | 1 | 0 | 1 | 1 | 0 |
| 7 | 1 | 1 | 0 | 1 | 0 |
| 8 | 1 | 1 | 1 | 1 | 1 |

Estas son las tablas de verdad para el comparador (=, >, <, repectivamente):

| | Tabla de Verdad | | |
|---|---|---|---|
| | A | B | S |
| 1 | 0 | 0 | 1 |
| 2 | 0 | 1 | 0 |
| 3 | 1 | 0 | 0 |
| 4 | 1 | 1 | 1 |

| | Tabla de Verdad | | |
|---|---|---|---|
| | A | B | S |
| 1 | 0 | 0 | 0 |
| 2 | 0 | 1 | 0 |
| 3 | 1 | 0 | 1 |
| 4 | 1 | 1 | 0 |

| | Tabla de Verdad | | |
|---|---|---|---|
| | A | B | S |
| 1 | 0 | 0 | 0 |
| 2 | 0 | 1 | 0 |
| 3 | 1 | 0 | 1 |
| 4 | 1 | 1 | 0 |

## B. Mapas de Karnaugh

| Salida (S) | | | | |
|---|---|---|---|---|
| A/BC | 00 | 01 | 11 | 10 |
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 1 | 0 |

| Salida (Cout) | | | | |
|---|---|---|---|---|
| A/BC | 00 | 01 | 11 | 10 |
| 0 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 1 | 1 |

| Mapa de Karnauh | | |
|---|---|---|
| A/B | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

| Mapa de Karnauh | | |
|---|---|---|
| A/B | 0 | 1 |
| 0 | 0 | 0 |
| 1 | 1 | 0 |

| Mapa de Karnauh | | |
|---|---|---|
| A/B | 0 | 1 |
| 0 | 0 | 1 |
| 1 | 0 | 0 |

## C. Funciones Lógicas

| SoP |
|---|
| A'B'C + A'BC' + AB'C' + ABC |
| A⊕B⊕C |

| SoP |
|---|
| BC + AC + AB |
| AB + C(A+B) |

| SoP |
|---|
| $A'B' + AB$ |
| A⊕B |

| SoP |
|---|
| $AB'$ |

| SoP |
|---|
| $A'B$ |

## D. Programación en Gowin con verilog

**Archivo MAIN.v:**

```
1  //Module Main Calculator
2  module CALCULADORA(
3      // Inputs and outputs declaration
4      input [3:0] A,
5      input [3:0] B,
6      input [1:0] Sel,
7      output [7:0] Y
8  );
9
10     //Internal wires
11     wire [7:0] Sum, Sub, Mul;
12
13     //Instantiate Adder
14     ADDER_4BITS ADDER_IC (
15         .A(A),
16         .B(B),
17
18         .S(Sum[3:0]),
19         .Cout(Sum[4])
20     );
21
22     //Instantiate Subtractor
23     SUBTRACTOR_4BITS SUBTRACTOR_IC (
24         .A(A),
25         .B(B),
26
27         .S(Sub[3:0]),
28         .Bout(Sub[4])
29     );
30
31     //Instantiate Multiplier
32     MULTIPLIER_4BITS MULTIPLIER_IC (
33         .A(A),
34         .B(B),
35
36         .P(Mul)
37     );
38
39     //Instantiate 8 bits Multiplexer
40     MULTIPLEXER_8BITS_4TO1 #( .Gnd(0) ) MUX_IC (
41         .D0(Sum),
42         .D1(Sub),
43         .D2(Mul),
44         .S(Sel),
45         .Y(Y)
46     );
47 endmodule
```

**Fulladder:**

```
//Module FULL ADDER
module FULL_ADDER(
    // Inputs and outputs declaration
    input A,B,Cin,
    output S, Cout
);

    //Internal wires
    wire S1,C1,C2;

    //Full Adder Declarations
    xor X1(S1,A,B);
    xor X2(S,S1,Cin);
    and A1(C1,A,B);
    and A2(C2,S1,Cin);
    or O1(Cout,C1,C2);
endmodule
```

**Adder:**

```verilog
//Module ADDER 4 BITS
module ADDER_4BITS #(parameter Cin = 0)(
    // Inputs and outputs declaration
    input [3:0] A,
    input [3:0] B,
    output [3:0] S,
    output Cout
);

    //Internal wires
    wire C1,C2,C3;

    //4 Bits Adder Declarations
    FULL_ADDER FA0 (
        .A(A[0]),
        .B(B[0]),
        .Cin(Cin),

        .S(S[0]),
        .Cout(C1)
    );

    FULL_ADDER FA1 (
        .A(A[1]),
        .B(B[1]),
        .Cin(C1),

        .S(S[1]),
        .Cout(C2)
    );

    FULL_ADDER FA2 (
        .A(A[2]),
        .B(B[2]),
        .Cin(C2),

        .S(S[2]),
        .Cout(C3)
    );

    FULL_ADDER FA3 (
        .A(A[3]),
        .B(B[3]),
        .Cin(C3),

        .S(S[3]),
        .Cout(Cout)
    );
endmodule
```

**Subtractor:**

```verilog
//Module SUBTRACTOR 4 BITS
module SUBTRACTOR_4BITS #(parameter Bin = 1)
    // Inputs and outputs declaration
    input [3:0] A,
    input [3:0] B,
    output [3:0] S,
    output Bout
);

    //Internal wires
    wire [3:0] B_inv;
    wire SA;

    //Invert B bits
    NOT NOT_IC (
        .NE1(B[0]),
        .NE2(B[1]),
        .NE3(B[2]),
        .NE4(B[3]),

        .NS1(B_inv[0]),
        .NS2(B_inv[1]),
        .NS3(B_inv[2]),
        .NS4(B_inv[3])
    );

    //4 Bits Subtractor Declarations
    ADDER_4BITS #( .Cin(Bin) ) ADDER_IC (
        .A(A),
        .B(B_inv),

        .S(S),
        .Cout(SA)
    );

    not CN1(Bout, SA);
endmodule
```

**Multiplier:**

```verilog
//Module Multiplier 4 BITS
module MULTIPLIER_4BITS(
    input  [3:0] A,
    input  [3:0] B,
    output [7:0] P
);

    //--------------------------------------------
    // 1. Productos parciales (bitwise AND)
    //--------------------------------------------
    wire PP0_0, PP0_1, PP0_2, PP0_3;
    wire PP1_0, PP1_1, PP1_2, PP1_3;
    wire PP2_0, PP2_1, PP2_2, PP2_3;
    wire PP3_0, PP3_1, PP3_2, PP3_3;

    and (PP0_0, A[0], B[0]);
    and (PP0_1, A[1], B[0]);
    and (PP0_2, A[2], B[0]);
    and (PP0_3, A[3], B[0]);

    and (PP1_0, A[0], B[1]);
    and (PP1_1, A[1], B[1]);
    and (PP1_2, A[2], B[1]);
    and (PP1_3, A[3], B[1]);

    and (PP2_0, A[0], B[2]);
    and (PP2_1, A[1], B[2]);
    and (PP2_2, A[2], B[2]);
    and (PP2_3, A[3], B[2]);

    and (PP3_0, A[0], B[3]);
    and (PP3_1, A[1], B[3]);
    and (PP3_2, A[2], B[3]);
    and (PP3_3, A[3], B[3]);

    //--------------------------------------------
    // 2. Suma de productos parciales
    //    (Etapa 1: PP0 y PP1)
    //--------------------------------------------
    wire [3:0] S1;
    wire C1_0, C1_1, C1_2, C1; // carries internos

    assign P[0] = PP0_0; // primer bit del producto

    // Suma desplazada: (PP0 << 1) + PP1
    FULL_ADDER FA1_0 (
        .A(PP1_0),
        .B(PP0_1),
        .Cin(1'b0),

        .S(S1[0]),
        .Cout(C1_0)
    );
    FULL_ADDER FA1_1 (
        .A(PP1_1),
        .B(PP0_2),
        .Cin(C1_0),

        .S(S1[1]),
        .Cout(C1_1)
    );
    FULL_ADDER FA1_2 (
        .A(PP1_2),
        .B(PP0_3),
        .Cin(C1_1),

        .S(S1[2]),
```

**MUX:**

```verilog
//Module Multiplixer 4:1
module MULTIPLEXER_4TO1 #(parameter Gnd = 0)(
    // Inputs and outputs declaration
    input [3:0] D,
    input [1:0] S,
    output Y
);

    //Internal wires
    wire w0, w1, w2, w3, w4, w5, w6, w7;

    //Not gates for select lines
    NOT N1 (
        .NE1(S[0]),
        .NS1(w0)
    );

    NOT N2 (
        .NE1(S[1]),
        .NS1(w1)
    );

    //AND gates for data lines and select lines
    and A1 (w2, D[0], w1, w0);
    and A2 (w3, D[1], w1, S[0]);
    and A3 (w4, D[2], S[1], w0);
    and A4 (w5, Gnd, S[1], S[0]);

    //OR gate to combine the outputs
    or O1 (Y, w2, w3, w4, w5);
endmodule
```

**MUX para el bus completo (8 bits):**

```
//Module Multiplxer for 8 bits
module MULTIPLEXER_8BITS_4TO1 #(parameter Gnd = 0)(
    // Inputs and outputs declaration
    input [7:0] D0,
    input [7:0] D1,
    input [7:0] D2,
    input [1:0] S,
    output [7:0] Y
);

    //Instantiate 8 multiplexers for each bit
    MULTIPLEXER_4TO1 #( .Gnd(Gnd) ) MUX0 (
        .D({Gnd, D2[0], D1[0], D0[0]}),
        .S(S),
        .Y(Y[0])
    );

    MULTIPLEXER_4TO1 #( .Gnd(Gnd) ) MUX1 (
        .D({Gnd, D2[1], D1[1], D0[1]}),
        .S(S),
        .Y(Y[1])
    );

    MULTIPLEXER_4TO1 #( .Gnd(Gnd) ) MUX2 (
        .D({Gnd, D2[2], D1[2], D0[2]}),
        .S(S),
        .Y(Y[2])
    );

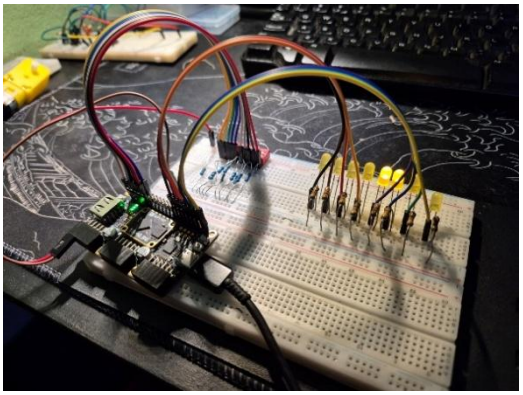
    MULTIPLEXER_4TO1 #( .Gnd(Gnd) ) MUX3 (
        .D({Gnd, D2[3], D1[3], D0[3]}),
        .S(S),
        .Y(Y[3])
    );

    MULTIPLEXER_4TO1 #( .Gnd(Gnd) ) MUX4 (
        .D({Gnd, D2[4], D1[4], D0[4]}),
        .S(S),
        .Y(Y[4])
    );

    MULTIPLEXER_4TO1 #( .Gnd(Gnd) ) MUX5 (
        .D({Gnd, D2[5], D1[5], D0[5]}),
        .S(S),
        .Y(Y[5])
    );

    MULTIPLEXER_4TO1 #( .Gnd(Gnd) ) MUX6 (
        .D({Gnd, D2[6], D1[6], D0[6]}),
        .S(S),
        .Y(Y[6])
    );
    MULTIPLEXER_4TO1 #( .Gnd(Gnd) ) MUX7 (
        .D({Gnd, D2[7], D1[7], D0[7]}),
        .S(S),
        .Y(Y[7])
    );
endmodule
```

## VI. FOTOS DEL CIRCUITO EN FUNCIONAMIENTO



**Explicación Breve sobre funcionamiento general del circuito y la estrategia de selección de la operación:**

El sistema combina cuatro módulos funcionales: sumador, restador, multiplicador y comparador, cuyas salidas convergen en un multiplexor 4:1 controlado por dos bits de selección. Dependiendo del valor de selección, los LEDs muestran el resultado correspondiente a la operación aritmética o a la comparación de las entradas.

## VII. CONCLUSION

La práctica permitió comprender de forma integral el proceso de diseño combinacional aritmético, desde la teoría booleana hasta la implementación en hardware real.

Se logró construir un sistema funcional que realiza operaciones de suma, resta y multiplicación de 4 bits, integradas mediante un multiplexor controlado, demostrando la potencia del diseño modular.

La segunda parte de la práctica amplió el sistema aritmético inicial integrando un módulo comparador de 5 bits, demostrando la escalabilidad del diseño modular en Verilog.

Se logró implementar un circuito que no solo realiza operaciones aritméticas, sino que también evalúa la relación lógica entre dos operandos, reforzando los conceptos de jerarquía y reutilización de módulos.
La simulación en Multisim y la verificación en FPGA confirmaron el correcto funcionamiento de las nuevas funciones añadidas, manteniendo la coherencia entre la teoría y la práctica.

En conclusión, esta ampliación permitió comprender cómo combinar operaciones aritméticas y lógicas en un solo sistema digital, consolidando el aprendizaje en lógica combinacional y diseño estructural.

## VIII. REFERENCIAS


https://en.wikipedia.org/wiki/Digital_electronics
https://naylampmechatronics.com/blog/39_resistencias-pull-up-y-pull-down.html
https://es.wikipedia.org/wiki/Sistema_de_numeraci%C3%B3n
https://es.wikipedia.org/wiki/C%C3%B3digo_binario
https://es.slideshare.net/slideshow/sistemas-de-codigos-en-la-electronica-digital/158426000